DoubleML
Difference-in-Differences

DoubleML Workshop

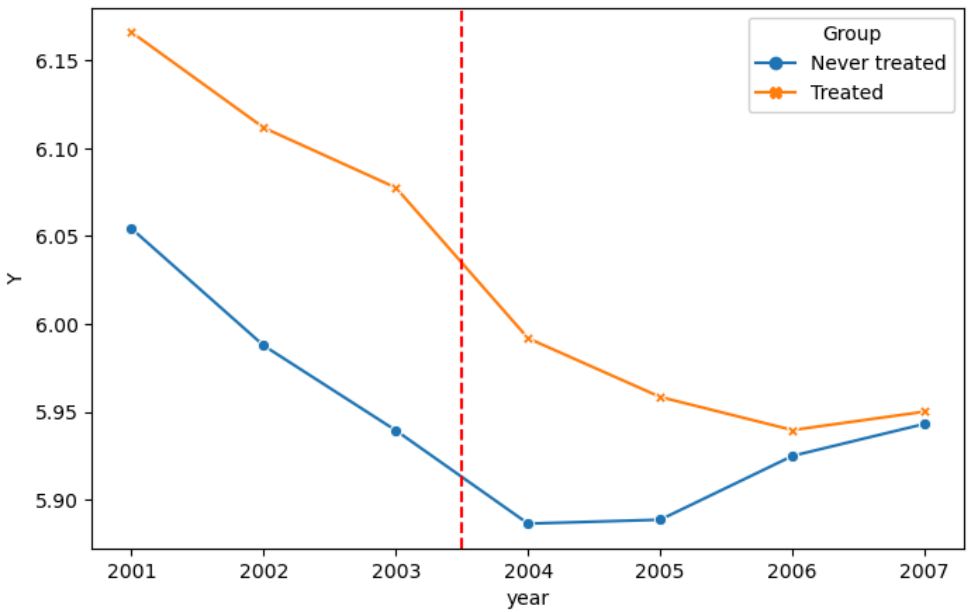
February 27 & March 06, 2026

Economic AI

Motivation & Basics

Motivation

- So far: Cross-sectional data
  - At a given point in time t, we observe n individuals (i.i.d)
- Now: Panel data and repeated cross-sectional data
  - We observe data for individuals at multiple points in time
- How can we estimate a causal effect in such a setting?



Data Types

Panel Data

Observe a subsample of individuals over a time period

Example:

Business to Business

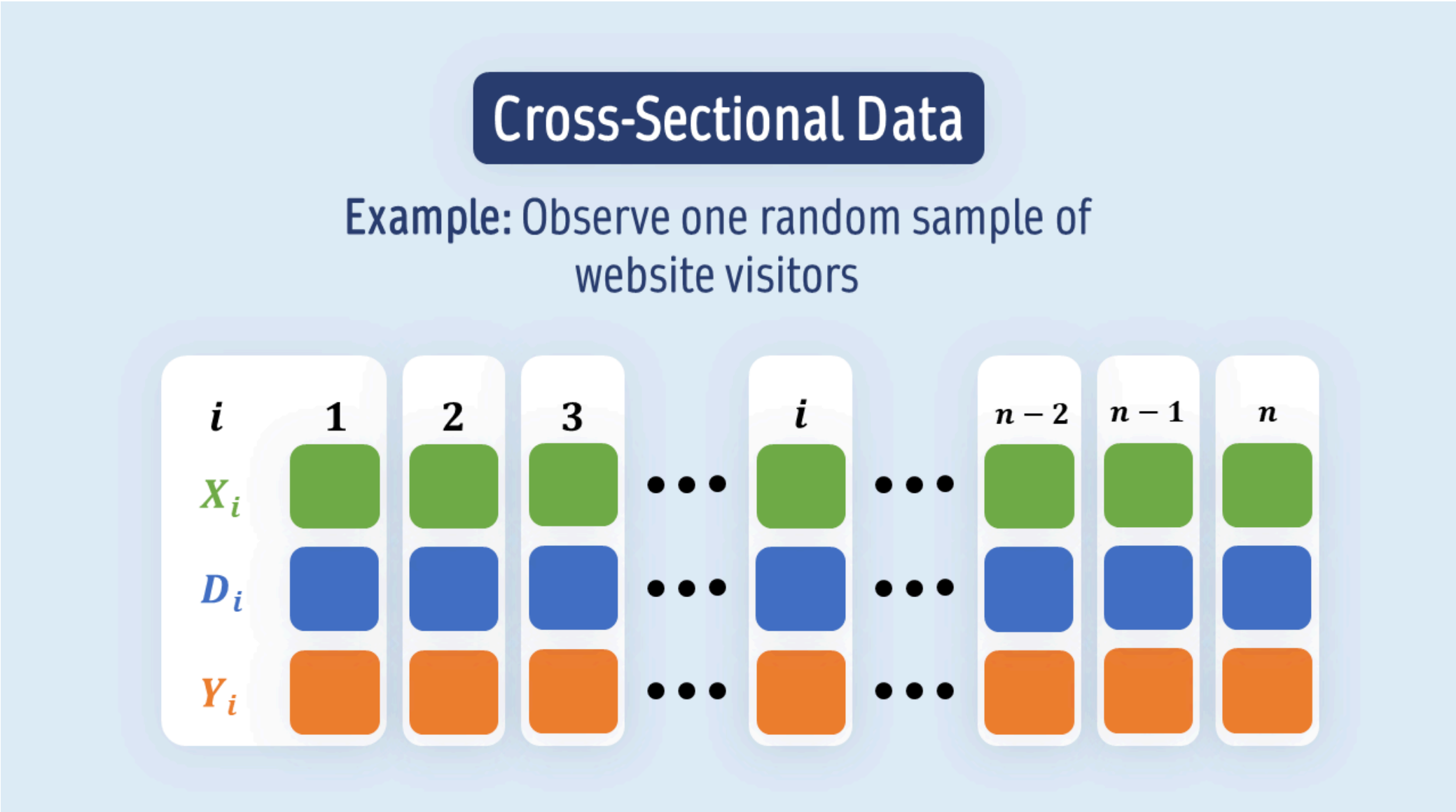
Repeated Cross Sections

At each time, observe a subsample of individuals

Example:

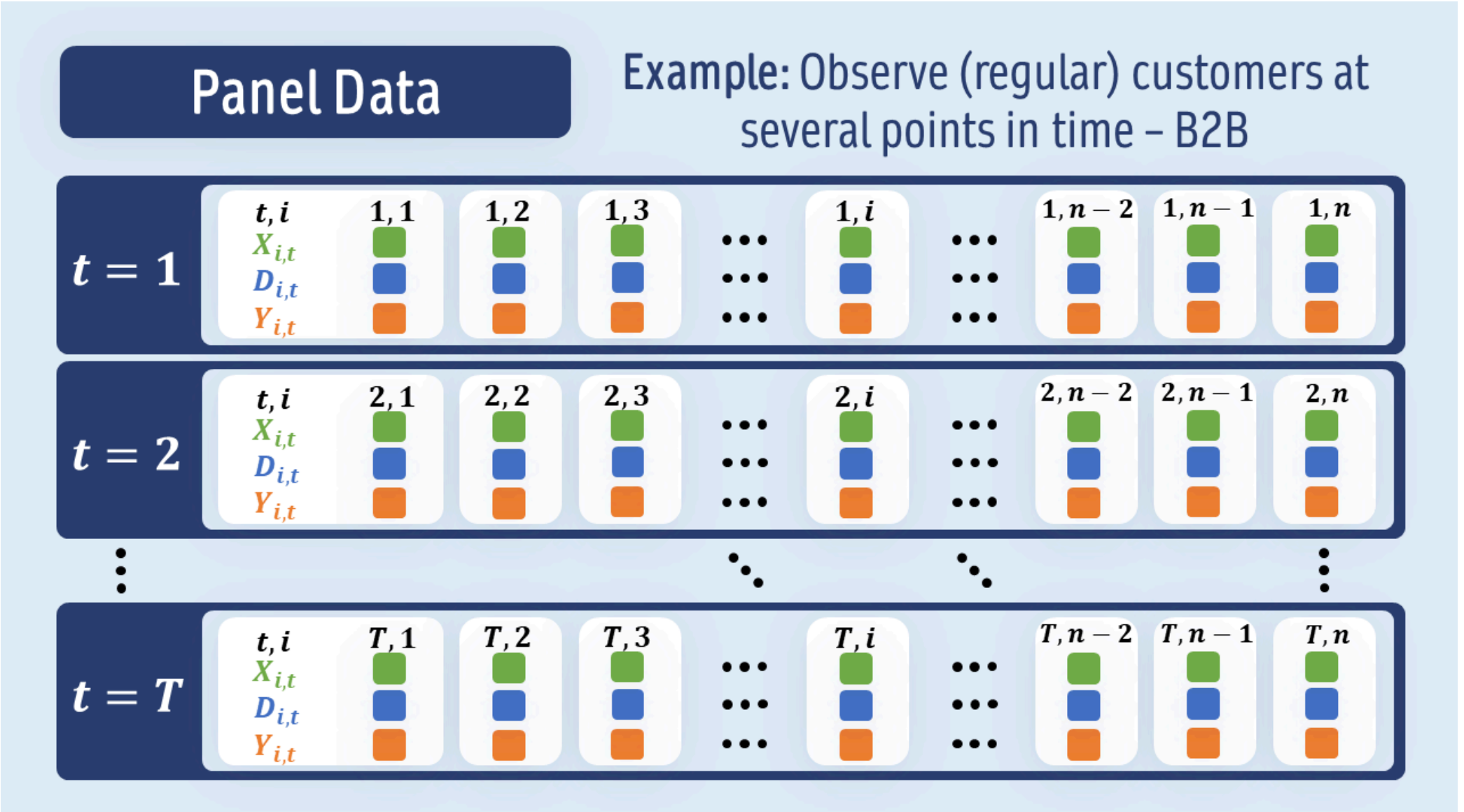
Business to Consumer

Motivation: Types of Data



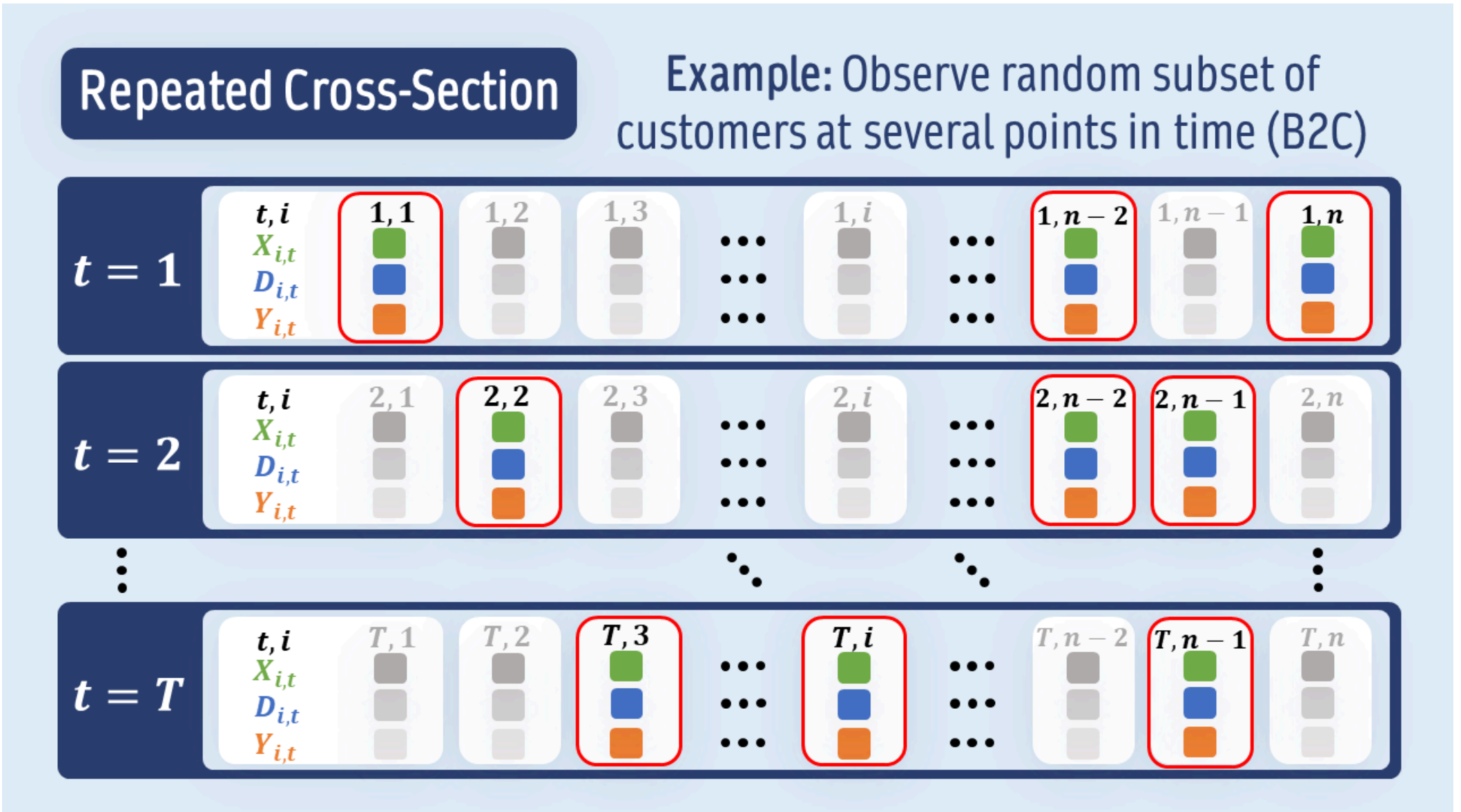
Cross-sectional data

Motivation: Types of Data



Panel data

Motivation: Types of Data

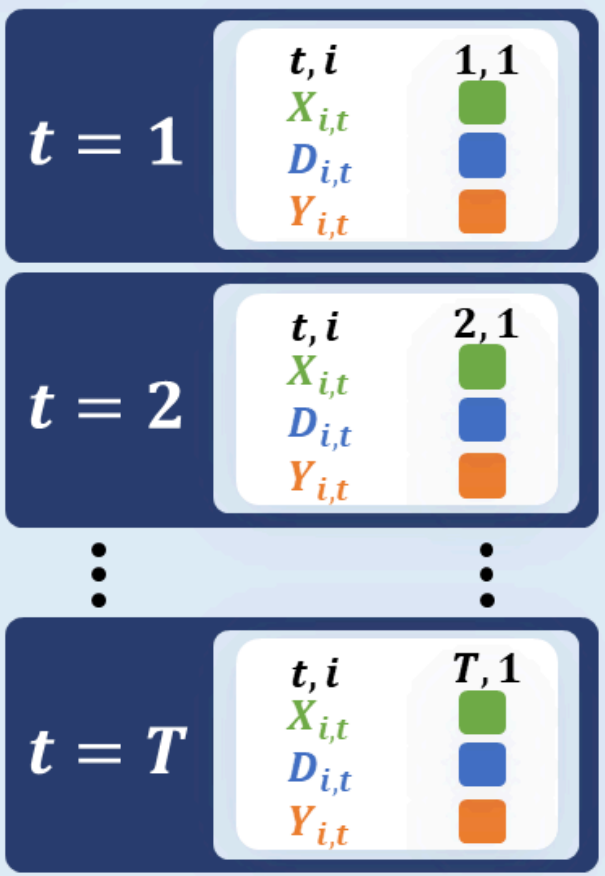


Repeated cross section

Motivation: Types of Data

Time Series

Example: Observe sales in one market over multiple periods



Time series

What is Difference-in-Differences (DiD)?

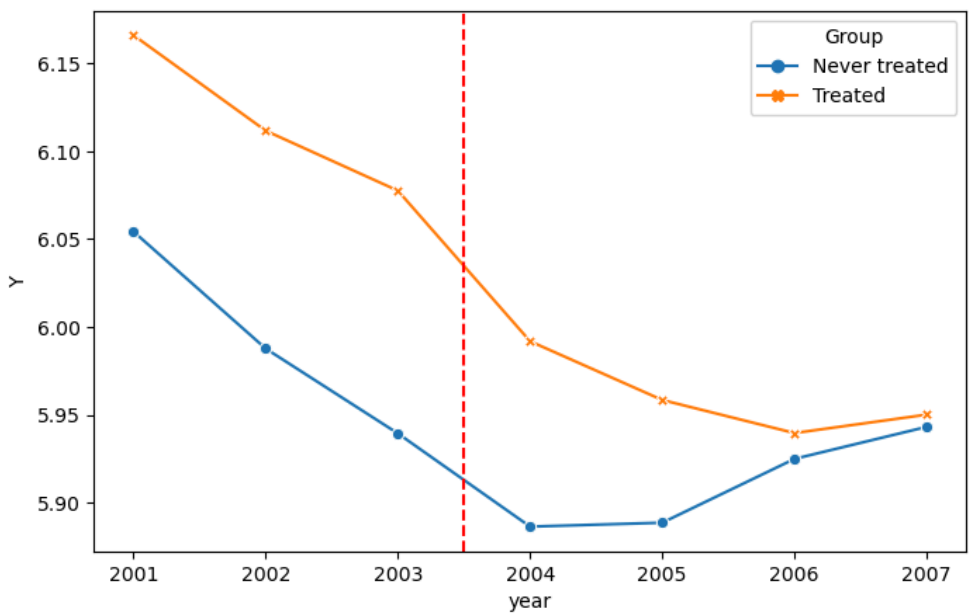
- A popular quasi-experimental design to estimate the **causal effect** of an intervention or treatment.
- Compares the change in outcomes over time between a **treated group** and a **control group**.

Core Idea: The control group's change in outcome provides a proxy for what would have happened to the treated group in the absence of treatment.

The DiD Setup: A Visual Introduction

Imagine tracking an outcome (e.g., youth unemployment) for two groups:

- One group receives a treatment (e.g., minimum wage increase).
- The other does not (control).



The DiD estimator captures the difference in the “differences” (changes over time) between these groups.

Notation: The 2x2 DiD Setting

Let's define the key variables for a simple DiD setup with two groups and two time periods:

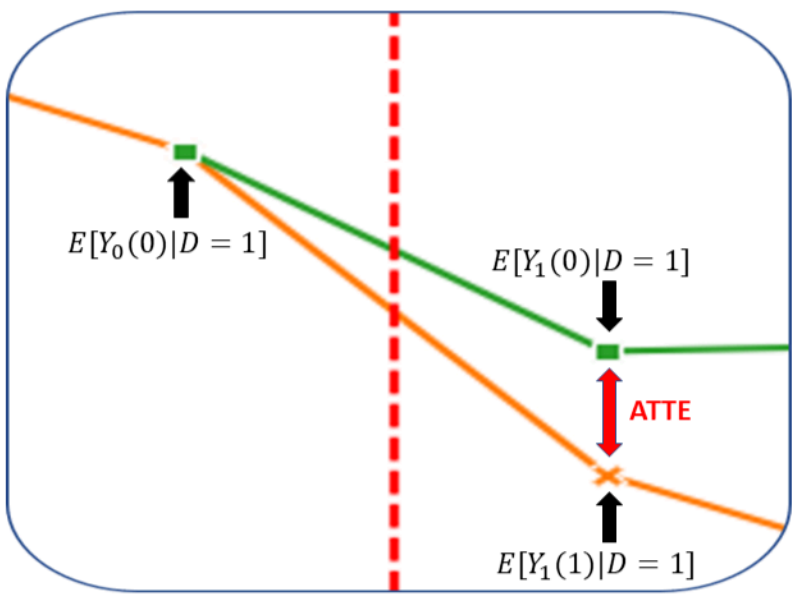
- **Time periods**:
  - $t = 0$: Pre-treatment period.
  - $t = 1$: Post-treatment period.
- **Outcome**: Y_t is the outcome of interest at time t.
- **Treatment Indicator**: D is a binary variable.
  - $D = 1$ if a unit belongs to the group that gets treated between $t = 0$ and $t = 1$.
  - $D = 0$ if a unit belongs to the control group (not treated).
- **Potential Outcomes**:
  - $Y_t(1)$: Potential outcome at time t if the unit *is treated* by time t.
  - $Y_t(0)$: Potential outcome at time t if the unit *is not treated* by time t.

Causal Estimand: ATT

The primary quantity of interest in a DiD study is often the **Average Treatment Effect on the Treated (ATT)**:

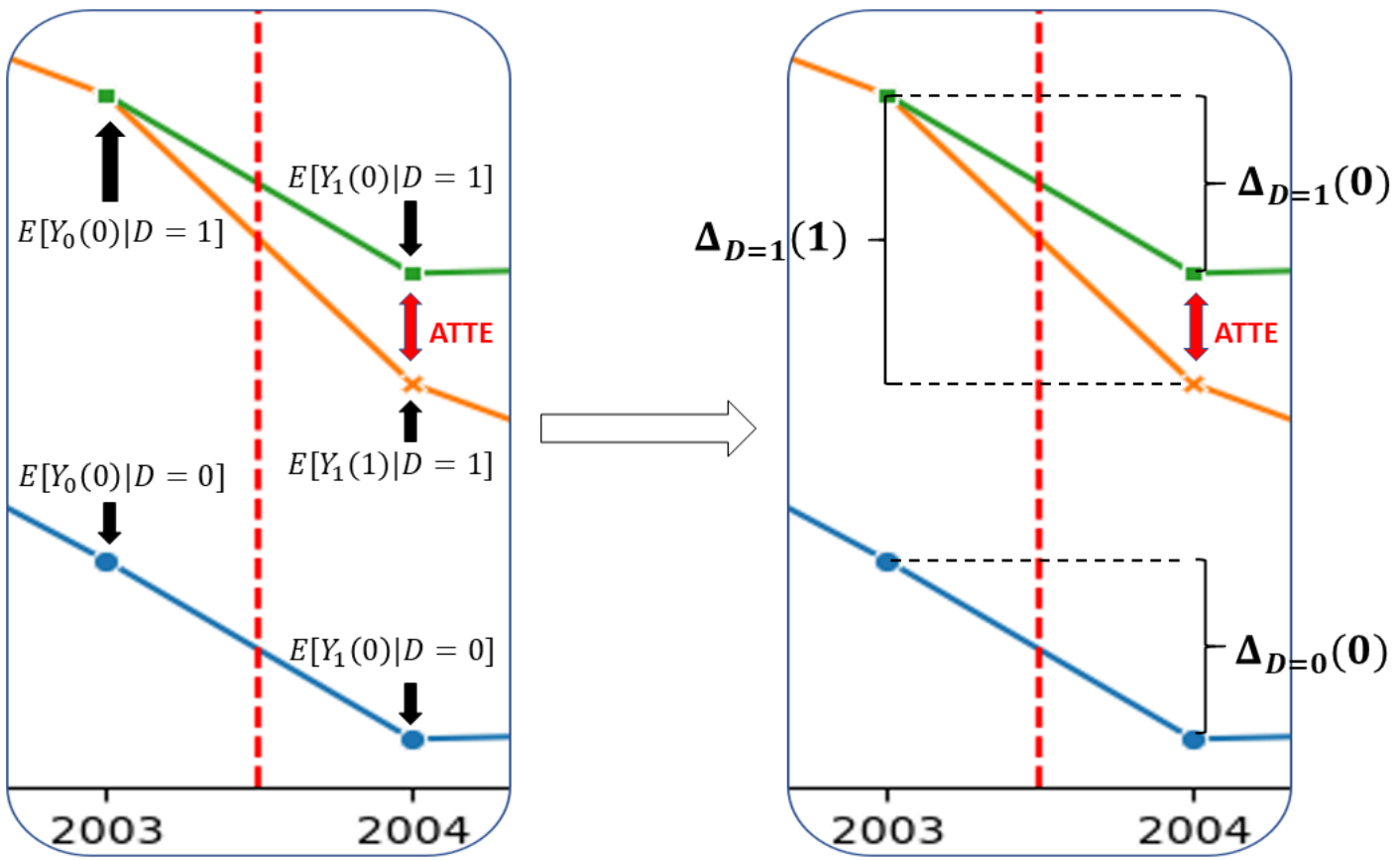
$$\theta_0 := \mathbb{E}[Y_1(1) - Y_1(0)|D = 1]$$

Or visually (the green line represents counterfactuals):



The Parallel Trends Assumption

- In the **absence of the treatment**, the average change in the outcome for the treated group would have been the same as the average change in the outcome for the control group.



The green line (counterfactual trend) is parallel to the blue line (control group trend).

Identifying ATT under Parallel Trends

- Under the crucial **Parallel Trends Assumption**, the ATT can be identified using observable data.
- The identification relies on comparing the change in outcomes for the treated group to the change in outcomes for the control group:

$$\theta_0 = \underbrace{(\mathbb{E}[Y_1|D=1] - \mathbb{E}[Y_0|D=1])}_{\text{Change for Treated Group}} - \underbrace{(\mathbb{E}[Y_1|D=0] - \mathbb{E}[Y_0|D=0])}_{\text{Change for Control Group}}$$

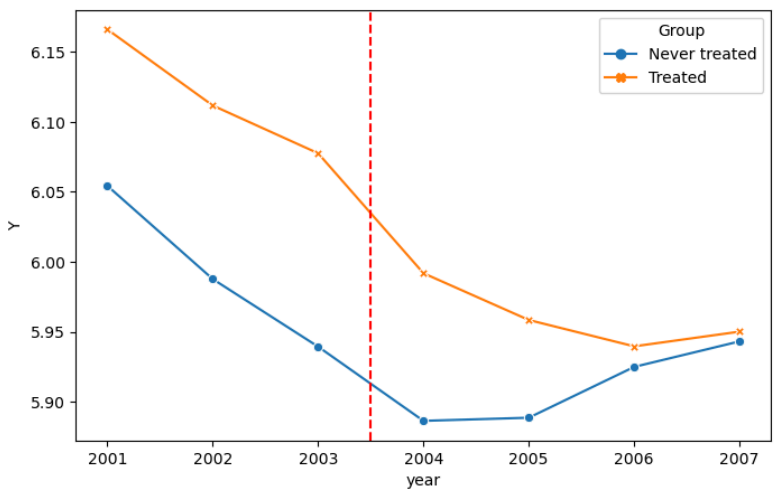
But is this assumption reasonable? How do we know the control group would have followed the same trend as the treated group?

Multiple Time Periods

Motivation

Panel Data: Observes the *same units* (e.g., individuals, firms, states) over *multiple time periods*.

- Treatment effects can vary:
  - They might not be constant post-intervention.
  - Effects can evolve dynamically across multiple time periods.
- Multiple pre-treatment periods are valuable:
  - Allow examination of pre-existing outcome trends.
  - Crucial for placebo tests to assess the parallel trends assumption.
- Potentially multiple groups being treated at different times (staggered adoption)



Setting: Panel Data Notation

Following Callaway and Sant'Anna (2021) and the DoubleML documentation, we define:

- $Y_{i,t}$: Outcome for unit i at time t.
- G_i: Group indicator, denoting the period when unit i is first treated. $G_i = \infty$ if unit i is never treated.
- $D_{i,t}$: Treatment indicator for unit i at time t. $D_{i,t} = 1$ if unit i is treated at time t, and 0 otherwise.
  - Staggered Adoption: $D_{i,t} = \mathbf{1}\{G_i \leq t\}$.
- X_i: Vector of pre-treatment covariates for unit i.
- $\square$: Set of time periods, e.g., $\square = \{1, \ldots, T_{max}\}$.

Causal Estimands in Multi-Period DiD

With multiple periods and groups, the focus shifts from a single ATT to ATTs for specific **group-time combinations**, denoted $ATT(\mathrm{g}, t)$.

$$ATT(\mathrm{g}, t) = \mathbb{E}[Y_t(\mathrm{g}) - Y_t(0) | G_i = \mathrm{g}]$$

- $Y_t(\mathrm{g})$: Potential outcome at time t if unit i was first treated at period g.
- $Y_t(0)$: Potential outcome at time t if unit i had not been treated up to time t.

Key Difference from 2x2 DiD:

- Here, $ATT(\mathrm{g}, t)$ allows the treatment effect to vary depending on:
  - **Treatment group** g (i.e., when they started treatment).
  - **At which time period** t the effect is being measured.

This provides a much richer understanding of treatment effect dynamics.

Assumptions 1 & 2: Treatment Structure & Data

1. Irreversible Treatment:

- Once a unit (e.g., individual, firm) is exposed to the treatment, it remains treated for all subsequent time periods.
- *Formally*: For all units i and time periods $t = 2, \dots, \square$: If $D_{i,t-1} = 1$ (treated at $t-1$), then $D_{i,t} = 1$ (treated at t) almost surely.

2. Panel Data (Random Sampling):

- The observed data constitutes a random sample from the population of interest.
- *Formally*: The collection of data for each unit i,

$$(Y_{i,1}, \dots, Y_{i,\square}, X_i, G_i, D_{i,1}, \dots, D_{i,\square})_{i=1}^{n}$$

is independent and identically distributed (i.i.d.).

Assumption 3: Limited Treatment Anticipation

The treatment does not affect outcomes *before* it is implemented, or at least not too far in advance.

Formally:

- There is a known number of anticipation periods, $\delta \geq 0$.
- For any group g (first treated at period g) and any time $t < \mathrm{g} - \delta$:

$$\mathbb{E}[Y_{i,t}(\mathrm{g})|X_i, G_i = \mathrm{g}] = \mathbb{E}[Y_{i,t}(0)|X_i, G_i = \mathrm{g}] \quad a.s.$$

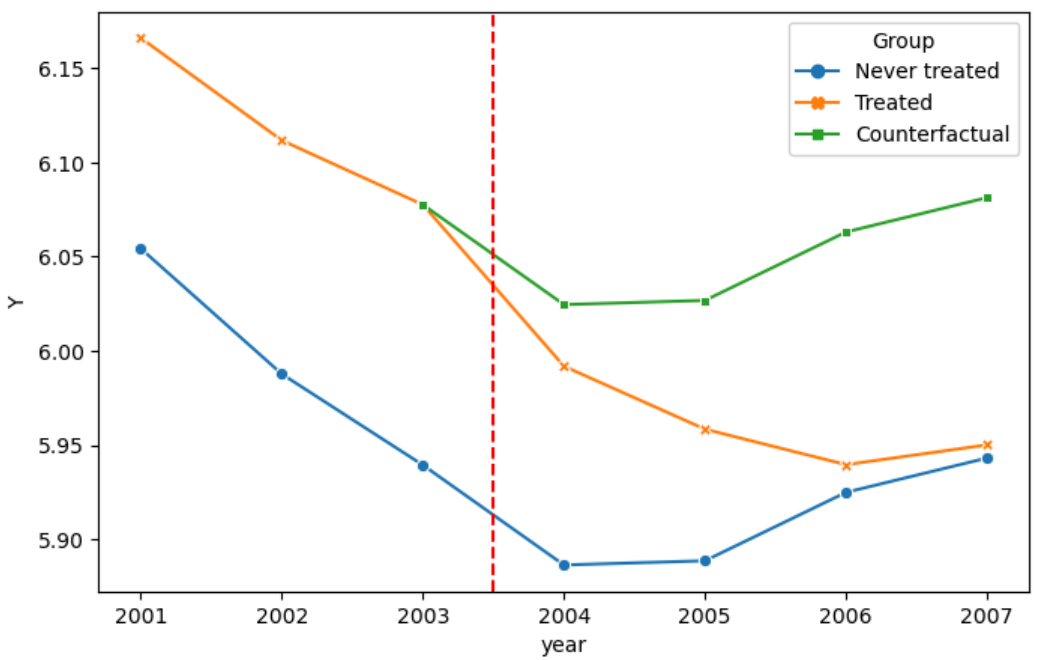
(The potential outcome under treatment g is the same as under no treatment, before $\mathrm{g} - \delta$).

`DoubleML` allows specifying `anticipation_periods` (δ), with default `anticipation_periods=0`, this means no anticipation.

Assumption 4: Conditional Parallel Trends

In the absence of treatment, and conditional on covariates X_i, the average change in outcomes for units in group g would have been the same as for units in the control group $\square$.

Or visualized (if it holds unconditional on covariates):



The green line (counterfactual trend) is parallel to the blue line (control group trend).

Assumption 4: Conditional Parallel Trends

Formally:

- For all groups $\mathrm{g} \in \square$ (set of treated groups) and relevant time periods $t \geq \mathrm{g} - \delta$ (post-treatment or close to it), and a pre-treatment period $t_{pre} < \mathrm{g} - \delta$:

$$\mathbb{E}[Y_{i,t}(0) - Y_{i,t_{pre}}(0) | X_i, G_i = \mathrm{g}] = \mathbb{E}[Y_{i,t}(0) - Y_{i,t_{pre}}(0) | X_i, G_i \in \square] \quad a.s.$$

- $Y_{i,t}(0)$ is the potential outcome if unit i is not treated up to time t.
- The control group $\square$ can be defined as:
  - `never_treated`: Units that are never treated (i.e., $G_i = \infty$).
  - `not_yet_treated`: Units that have not been treated by time t (i.e., $G_i > t$).

Assumption 5: (Weak) Overlap (Common Support)

For each treated unit i in group g, there should be a positive probability of observing a similar unit (conditional on covariates X_i) in the control group $\square$.

Formally:

Fore each time period $t = 2, \ldots, \square$ and each group $\mathrm{g} \in \square$ there exisits a $\epsilon > 0$ such that:

- $P(G_i = \mathrm{g}) > \epsilon$ for treated units i in group g.
- $P(G_i = \mathrm{g} | X_i, G_i = \mathrm{g} \text{ or } G_i \in \square) < 1 - \epsilon$ for treated units i in group g.

Identification

Under the previous assumptions, the Average Treatment Effect on the Treated (ATT)

$$ATT(\mathrm{g}, t_{\text{eval}}) = \mathbb{E}[Y_{t_{\text{eval}}}(\mathrm{g}) - Y_{t_{\text{eval}}}(0) | G_i = \mathrm{g}]$$

for group g at time t_{eval} can be identified.

More specifically, under a suitable choice of pre-treatment period let $\theta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}}$ be the solution of a linear moment condition[1]:

$$\mathbb{E}[\psi^a(W_i; \eta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}})\theta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}} + \psi^b(W_i; \eta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}})] = 0,$$

where $\eta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}}$ are nuisance elements which depend on the group g, pre-treatment period t_{pre}, and evaluation period t_{eval}.

If the assumptions (1-5) hold, then $\theta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}}$ identifies the ATT for group g at time t_{eval} (for any suitable pre-treatment period t_{pre}).

$$ATT(\mathrm{g}, t_{\text{eval}}) = \theta_{\mathrm{g},t_{\text{pre}},t_{\text{eval}}} =: ATT(\mathrm{g}, t_{\text{pre}}, t_{\text{eval}})$$

Nuisance Elements

The required nuisance elements[1] for the moment condition are:

- **Conditional Trend (g_0)**: Models the expected change in the outcome for untreated units (or units as if untreated) between a pre-treatment period t_{pre} and an evaluation period t_{eval}, conditional on covariates X_i. For a specific group g and control group $\square$:

$$g_0(X_i) \equiv g_{0,g,t_{pre},t_{eval}}(X_i) = \mathbb{E}[Y_{i,t_{eval}} - Y_{i,t_{pre}} | X_i, G_i \in \square]$$

- **Propensity Score (m_0)**: Models the probability that a unit belongs to a specific treatment group g (first treated at period g) at the evaluation time t_{eval}, conditional on covariates X_i, given that the unit is either in group g or in the control group $\square$.

$$m_0(X_i) \equiv m_{0,g,t_{eval}}(X_i) = P(G_i = g | X_i, G_i = g \text{ or } G_i \in \square)$$

Summary: Identification & Nuisance Elements

- **Goal**: Estimate $ATT(g, t)$, the average treatment effect for group g at time t.
- **Identification**: Relies on a set of assumptions (e.g., cond. parallel trends, limited anticiption, overlap).
- **Key Idea**: The ATT is identified via a moment condition.
- **Nuisance Elements** $\eta = (g_0, m_0)$:
  - $g_0(X_i)$: Expected outcome change from $Y_{t_{\text{eval}}} - Y_{t_{\text{pre}}}$ for untreated, conditional on X_i.
  - $m_0(X_i)$: Propensity of being in group g, conditional on X_i.
- **Estimation**: These nuisance functions are estimated using machine learning and cross-fitting, afterwards the score is solved to estimate $ATT(g, t_{\text{pre}}, t_{\text{eval}})$ is estimated.

DiD with DoubleML

Introduction

- The DoubleML package provides tools to implement DiD with **panel data** and **repeated cross-sections**.
- Accommodates complex settings:
  - Multiple groups and time periods (staggered adoption).
  - Covariate adjustment using machine learning to strengthen the parallel trends assumption.

This presentation focuses on the mimum wage example (for details, see Callaway and Sant'Anna (2021)):

- Setting up `DoubleMLPanelData` objects.
- Estimating Average Treatment Effects on the Treated (ATTs).
- Aggregating and visualizing results.

Example: Callaway & Sant'Anna (2021) Data

Let us take a look at the minimum wage example (for details, see Callaway and Sant'Anna (2021)). Start with data in a **long format**, where each row represents a unit-period observation.

▶ Code

	year	Groups	Y	lpop	lavg_pay	id	D	G	region_1	region_
2	2002	Treated	5.356586	9.623972	10.097120	8003	1	2007	False	False
3	2003	Treated	5.389072	9.620859	10.107611	8003	1	2007	False	False
4	2004	Treated	5.356586	9.626548	10.140337	8003	1	2007	False	False
5	2005	Treated	5.303305	9.637958	10.175497	8003	1	2007	False	False
6	2006	Treated	5.342334	9.633056	10.218590	8003	1	2007	False	False

The `DoubleMLPanelData` Object

To use DiD with DoubleML, data must be structured as a `DoubleMLPanelData` object.

Key arguments for `DoubleMLPanelData`:

- `data`: Pandas DataFrame in long format (unit-period observations).
- `y_col`: Name of the outcome variable column.
- `d_cols`: Name of the group variable column (indicating first treatment period).
- `t_col`: Name of the time period column.
- `id_col`: Name of the unit identifier column.
- `x_cols`: List of covariate column names (for conditional trend).

The `DoubleMLPanelData` Object

```python
from doubleml.data import DoubleMLPanelData

dml_panel_data = DoubleMLPanelData(
    df_subset,
    y_col="Y",
    d_cols="G",
    t_col="year",
    id_col="id",
    x_cols=["lpop", "lavg_pay", "region_2", "region_3", "region_4"],
)
print(dml_panel_data)
```

```
================== DoubleMLPanelData Object ==================

------------------ Data summary      ------------------
Outcome variable: Y
Treatment variable(s): ['G']
Covariates: ['lpop', 'lavg_pay', 'region_2', 'region_3', 'region_4']
Instrument variable(s): None
Time variable: year
Id variable: id
Static panel data: False
No. Unique Ids: 2491
No. Observations: 14946
------------------ DataFrame info    ------------------
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14946 entries, 0 to 14945
Columns: 12 entries, year to region_4
dtypes: bool(4), float64(3), int64(4), object(1)
memory usage: 992.6+ KB
```

Digression: Stacking Learners g

- Packages & Hyperparameters
- Pipeline for ml_g

```
Pipeline(steps=[('scaler', RobustScaler()),
                ('stack',
                 StackingRegressor(estimators=[('lgbm',
                                                LGBMRegressor(colsample_bytree=0.8,
                                                              learning_rate=0.05,
                                                              max_depth=4,
                                                              n_estimators=1000,
                                                              num_leaves=15,
                                                              random_state=42,
                                                              reg_alpha=1.0,
                                                              reg_lambda=1.0,
                                                              subsample=0.8,
                                                              verbosity=-1)),
                                               ('ridge',
                                                Ridge(random_state=42)),
                                               ('rf',
                                                RandomForestRegressor(max_depth=5,
                                                                      max_features='sqrt',
                                                                      min_samples_leaf=5,
                                                                      min_samples_split=10,
```

Digression: Stacking Learners m

- Analogously, we can use stacking for the propensity score model.

▶ Pipeline for ml_m

```
Pipeline(steps=[('scaler', RobustScaler()),
                ('stack',
                 StackingClassifier(estimators=[('lgbm',
                                                 LGBMClassifier(colsample_bytree=0.8,
                                                                learning_rate=0.05,
                                                                max_depth=4,
                                                                n_estimators=1000,
                                                                num_leaves=15,
                                                                random_state=42,
                                                                reg_alpha=1.0,
                                                                reg_lambda=1.0,
                                                                subsample=0.8,
                                                                verbosity=-1)),
                                                ('logreg',
                                                 LogisticRegression(max_iter=1000,
                                                                    penalty='l2',
                                                                    random_state=42)),
                                                ('rf',
                                                 RandomForestClassifier(max_depth=5,
                                                                        min_samples_leaf=5,
```

`DoubleMLDIDMulti` Class

- To get the treatment effects for each group G over all time periods, we pass the `DoubleMLPanelData` object we just created, in the `DoubleMLDIDMulti` class.

```python
from doubleml.did import DoubleMLDIDMulti

dml_obj = DoubleMLDIDMulti(
    dml_panel_data,
    ml_g=pipeline_g,
    ml_m=pipeline_m,
    gt_combinations="standard",
    control_group="never_treated",
    n_folds=5,
)
```

Effect Estimation

- Estimation via the `fit()` method.

```
dml_obj.fit()
print(dml_obj.summary)
```

```
                        coef   std err          t         P>|t|     2.5 %  \
ATT(2004,2002,2003)  0.016482  0.013558   1.215674  2.241093e-01 -0.010091
ATT(2004,2003,2004) -0.028045  0.019136  -1.465553  1.427702e-01 -0.065550
ATT(2004,2003,2005) -0.057046  0.020127  -2.834252  4.593313e-03 -0.096495
ATT(2004,2003,2006) -0.080302  0.019440  -4.130741  3.615958e-05 -0.118404
ATT(2004,2003,2007) -0.076512  0.022025  -3.473949  5.128594e-04 -0.119680
ATT(2005,2002,2003)  0.007787  0.011141   0.698923  4.846001e-01 -0.014050
ATT(2005,2003,2004)  0.024788  0.012029   2.060621  3.933917e-02  0.001211
ATT(2005,2004,2005) -0.055435  0.009278  -5.975186  2.298281e-09 -0.073619
ATT(2005,2004,2006) -0.114903  0.010374 -11.075696  0.000000e+00 -0.135236
ATT(2005,2004,2007) -0.169502  0.013635 -12.431286  0.000000e+00 -0.196227
ATT(2006,2002,2003)  0.047702  0.009756   4.889407  1.011402e-06  0.028580
ATT(2006,2003,2004)  0.022110  0.008903   2.483337  1.301579e-02  0.004660
ATT(2006,2004,2005)  0.018646  0.007094   2.628484  8.576644e-03  0.004742
ATT(2006,2005,2006) -0.008652  0.008096  -1.068611  2.852451e-01 -0.024521
ATT(2006,2005,2007) -0.049911  0.008136  -6.134898  8.521384e-10 -0.065857
ATT(2007,2002,2003)  0.014483  0.007140   2.028405  4.251895e-02  0.000489
ATT(2007,2003,2004)  0.014810  0.008360   1.771600  7.646098e-02 -0.001575
ATT(2007,2004,2005) -0.008363  0.006422  -1.302284  1.928194e-01 -0.020950
ATT(2007,2005,2006) -0.033107  0.007089  -4.670153  3.009758e-06 -0.047001
```

Detailed Summary

- The string representation the `DoubleMLDIDMulti` object includes more details.

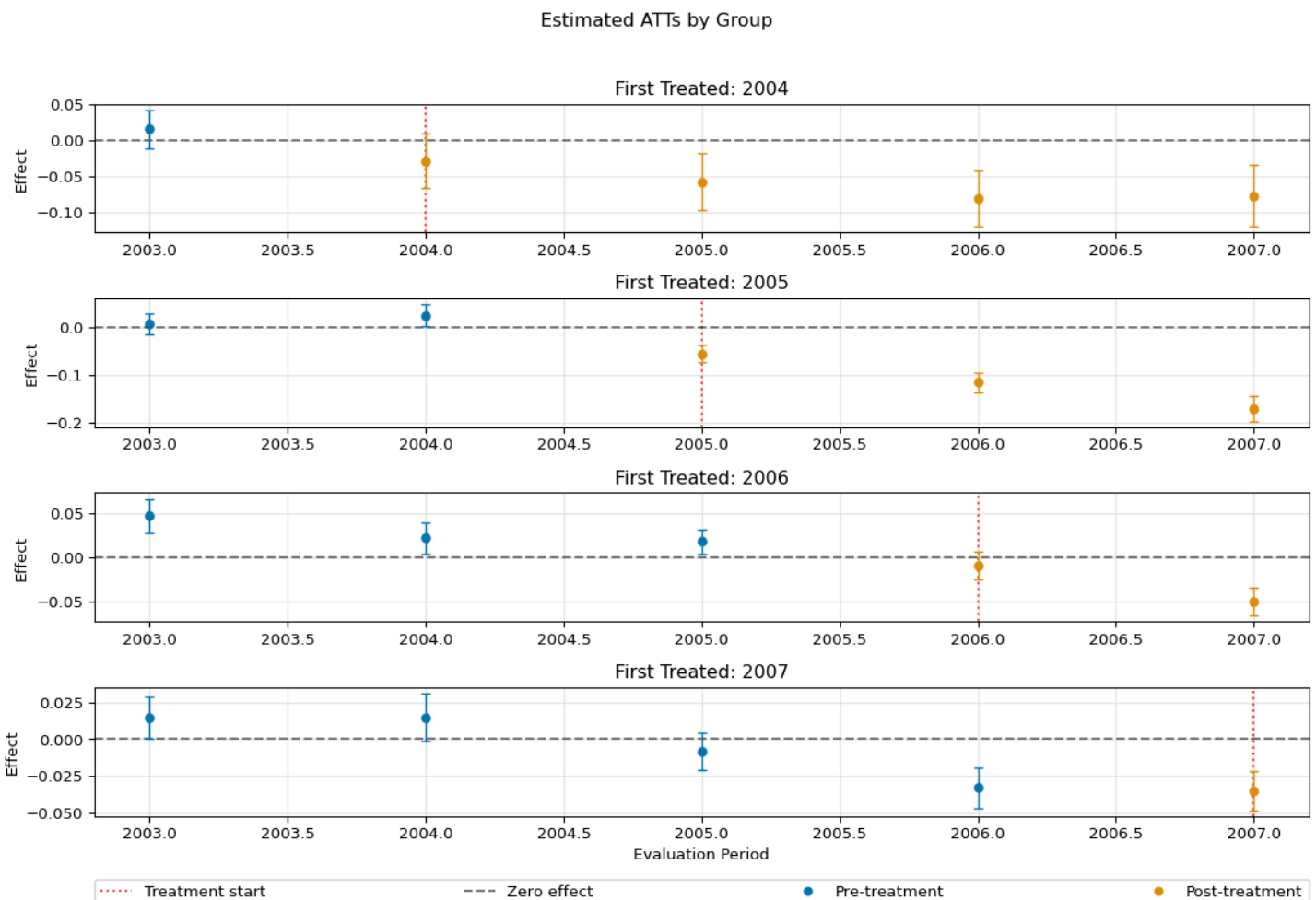
```
1 print(dml_obj)
```

```
================== DoubleMLDIDMulti Object ==================

------------------ Data summary      ------------------
Outcome variable: Y
Treatment variable(s): ['G']
Covariates: ['lpop', 'lavg_pay', 'region_2', 'region_3', 'region_4']
Instrument variable(s): None
Time variable: year
Id variable: id
Static panel data: False
No. Unique Ids: 2491
No. Observations: 14946
------------------ Score & algorithm ------------------
Score function: observational
Control group: never_treated
Anticipation periods: 0

------------------ Machine learner   ------------------
Learner ml_g: Pipeline(steps=[('scaler', RobustScaler()),
                ('stack',
```

Plotting Results

- Plot the event plot for the different groups.

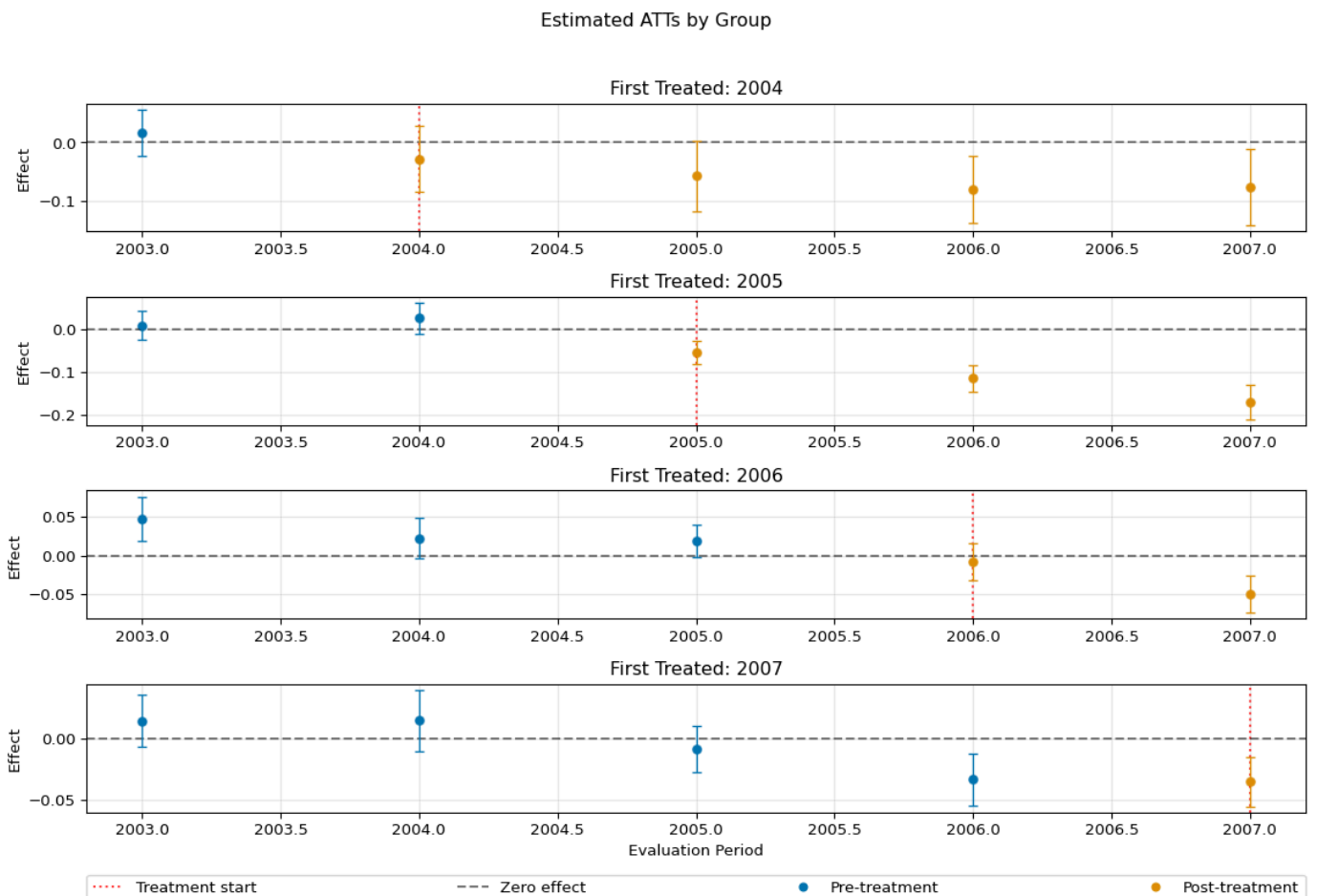
```
1  _ = dml_obj.plot_effects(joint = False)
```



Simultaneous Inference

- Joint confidence intervals can be obtained via the `bootstrap()` method.

```
dml_obj.bootstrap(n_rep_boot=5000)
_ = dml_obj.plot_effects(joint = True)
```



2x2 Submodels

- The different submodels can be accessed via the `modellist` attribute.

```
two_period_model = dml_obj.modellist[0]
print(two_period_model)
```

```
================== DoubleMLDIDBinary Object ==================

------------------ Data Summary      ------------------
Outcome variable: Y
Treatment variable(s): ['G']
Covariates: ['lpop', 'lavg_pay', 'region_2', 'region_3', 'region_4']
Instrument variable(s): None
Time variable: year
Id variable: id
Static panel data: False
No. Unique Ids: 2491
No. Observations: 14946

------------------ Score & Algorithm ------------------
Score function: observational\nTreatment group: 2004\nPre-treatment period: 2002\nEvaluation period:
2003\nControl group: never_treated\nAnticipation periods: 0\nEffective sample size: 1519

------------------ Machine Learner   ------------------
Learner ml_g: Pipeline(steps=[('scaler', RobustScaler()),
                ('stack',
```

Aggregated Effects

Aggregating Effects

Aggregated effects take the form

$$\theta_0 = \sum_{g \in \square} \sum_{t=2}^{\square} \omega_{g,t} ATT(g,t)$$

where $\omega_{g,t}$ are weights that can be specified (for details, see Callaway and Sant'Anna (2021)).

The following weighting schemes are available:

- **Group Aggregation**: Aggregates $ATT(g,t)$'s for each group g over relevant t.
- **Calendar Time Aggregation**: Aggregates $ATT(g,t)$'s for each calendar period t (based on group size).
- **Event Study Aggregation**: Aggregates $ATT(g,t)$'s by event time $e = t - g$.

Group Aggregation

- To aggregate effects by group, use the `aggregate()` method with `aggregation="group"`.

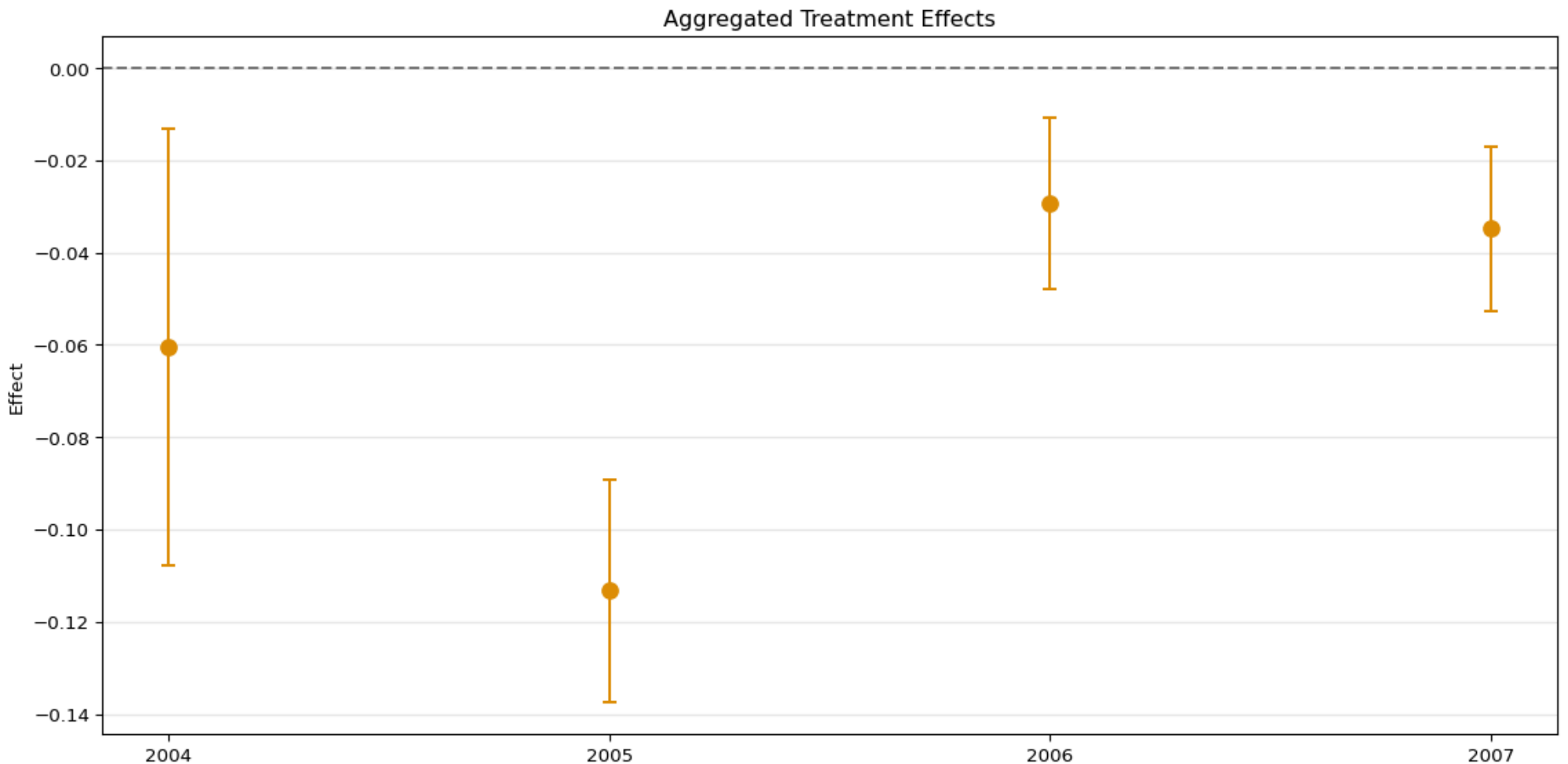
```
aggregated_group_obj = dml_obj.aggregate(aggregation="group")
print(aggregated_group_obj)
```

```
================== DoubleMLDIDAggregation Object ==================
 Group Aggregation

------------------ Overall Aggregated Effects ------------------
     coef  std err         t        P>|t|     2.5 %    97.5 %
-0.040503 0.005013 -8.079402 6.661338e-16 -0.050328 -0.030677
------------------ Aggregated Effects         ------------------
          coef   std err          t         P>|t|     2.5 %    97.5 %
2004 -0.060476  0.018188  -3.325081  8.839273e-04 -0.096124 -0.024829
2005 -0.113280  0.009271 -12.218477  0.000000e+00 -0.131451 -0.095109
2006 -0.029281  0.007118  -4.113702  3.893643e-05 -0.043233 -0.015330
2007 -0.034805  0.006831  -5.094998  3.487450e-07 -0.048194 -0.021416
------------------ Additional Information     ------------------
Score function: observational
Control group: never_treated
Anticipation periods: 0
```

Group Aggregation Plot

- The aggregated effects can be visualized using the `plot_effects()` method.

```
1  _ = aggregated_group_obj.plot_effects()
```



Eventstudy Aggregation

- To aggregate effects by event time, use the `aggregate()` method with `aggregation="eventstudy"`.

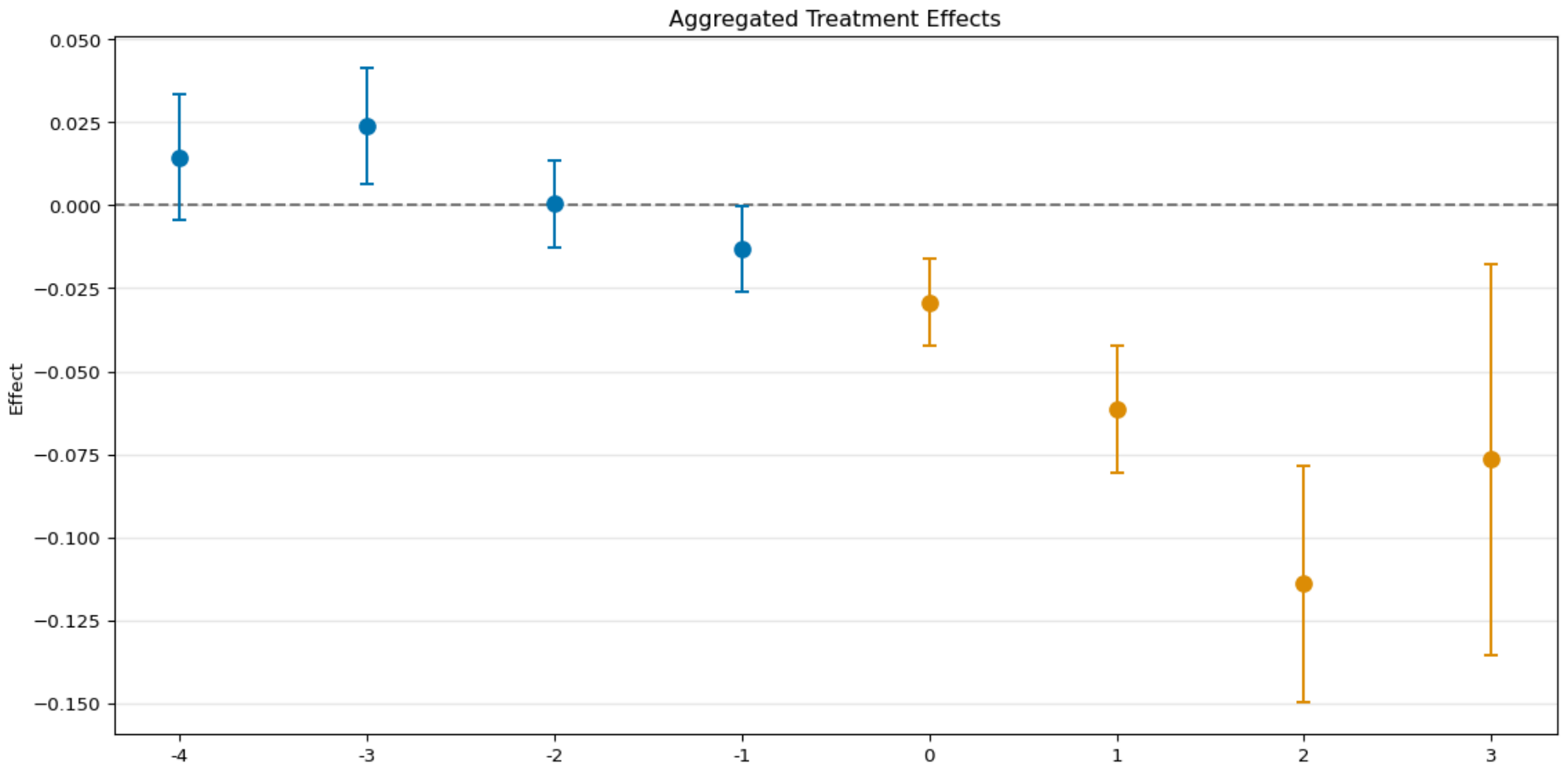
```
aggregated_eventstudy_obj = dml_obj.aggregate(aggregation="eventstudy")
print(aggregated_eventstudy_obj)
```

```
================== DoubleMLDIDAggregation Object ==================
 Event Study Aggregation

------------------ Overall Aggregated Effects ------------------
    coef  std err         t        P>|t|     2.5 %    97.5 %
-0.07034 0.010114 -6.955052 3.524292e-12 -0.090163 -0.050518
------------------ Aggregated Effects          ------------------
        coef   std err         t         P>|t|     2.5 %    97.5 %
-4  0.014483  0.007140  2.028405  4.251895e-02  0.000489  0.028477
-3  0.023738  0.006553  3.622401  2.918807e-04  0.010894  0.036582
-2  0.000411  0.004889  0.083998  9.330578e-01 -0.009172  0.009993
-1 -0.013153  0.004806 -2.736711  6.205684e-03 -0.022573 -0.003733
0  -0.029339  0.004911 -5.974533  2.307509e-09 -0.038964 -0.019714
1  -0.061486  0.007200 -8.539432  0.000000e+00 -0.075598 -0.047374
2  -0.114024  0.013321 -8.559750  0.000000e+00 -0.140133 -0.087916
3  -0.076512  0.022025 -3.473949  5.128594e-04 -0.119680 -0.033345
------------------ Additional Information     ------------------
Score function: observational
Control group: never_treated
Anticipation periods: 0
```

Eventstudy Aggregation Plot

- The aggregated effects can be visualized using the `plot_effects()` method.

```
1  _ = aggregated_eventstudy_obj.plot_effects()
```



Outlook

More Complex Scenarios

The previous example was a simplified introduction. `DoubleML` can handle more complex panel DiD settings:

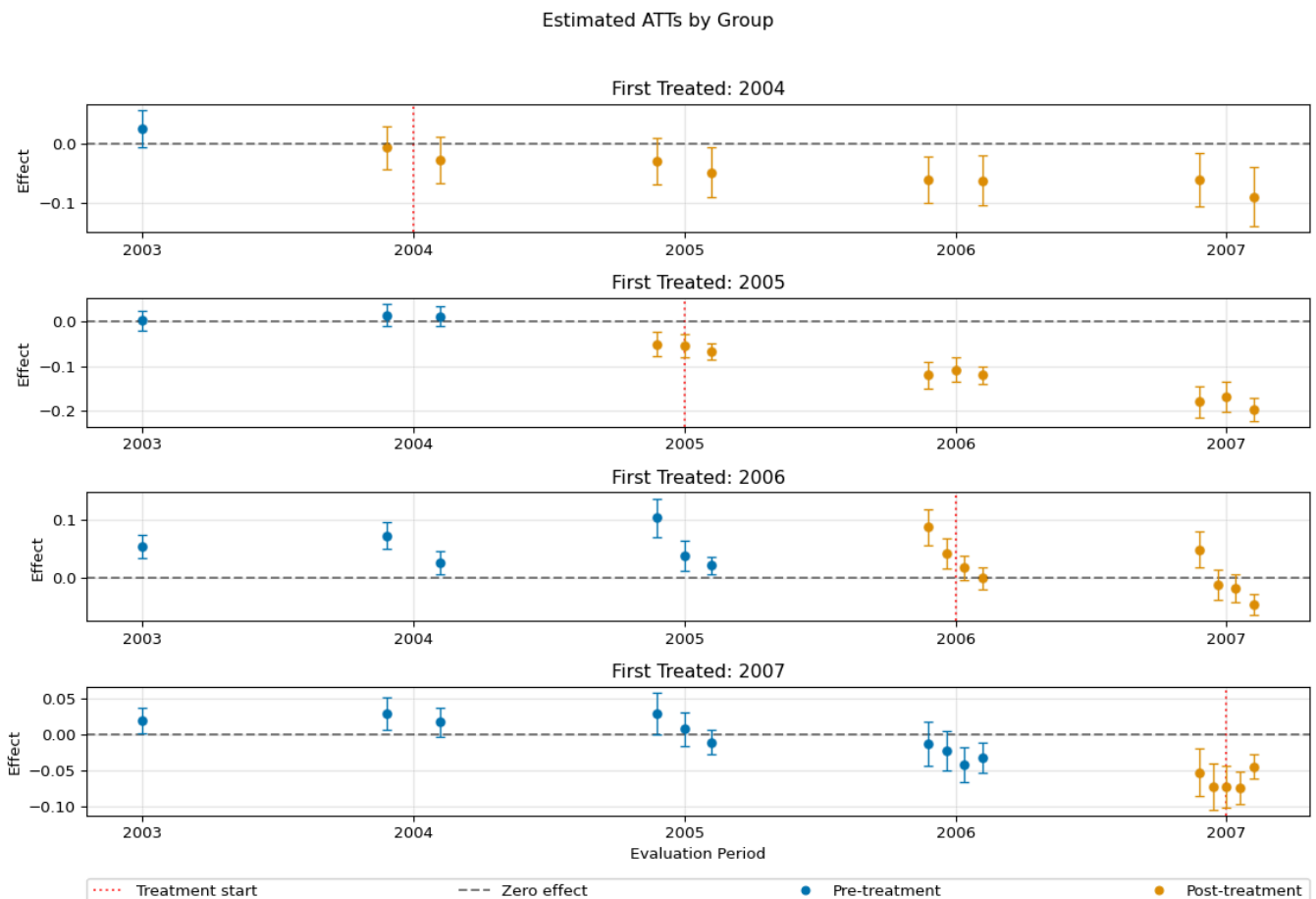
- **Varying `gt_combinations`**: Estimate effects for specific group-time pairs or all possible ones.
- **`datetime` time columns**: Work directly with date/time objects for the time variable.
- **Anticipation effects**: Account for units anticipating treatment via `anticipation_periods`.
- **Different control groups**: Use `not_yet_treated` as an alternative control group.
- **Sensitivity analysis**: Evaluate robustness with respect to unobserved confounders using `sensitivity_analysis`.

The Python: Panel Data with Multiple Time Periods example in the DoubleML Example Gallery a detailed guide for these more advanced features.

Different `gt_combinations`

- All possible group-time combinations are set by `gt_combinations="all"`.

▶ Code



Sensitivity Analysis

- Sensitivity with regard to unobserved confounders can be performed via the `sensitivity_analysis()` method.

```
dml_obj.sensitivity_analysis()
print(dml_obj.sensitivity_summary)
```

```
================== Sensitivity Analysis ==================

------------------ Scenario          ------------------
Significance Level: level=0.95
Sensitivity parameters: cf_y=0.03; cf_d=0.03, rho=1.0

------------------ Bounds with CI    ------------------
                      CI lower  theta lower     theta  theta upper  CI upper
ATT(2004,2002,2003) -0.026444    -0.004042  0.016482     0.037005  0.059486
ATT(2004,2003,2004) -0.079946    -0.048907 -0.028045    -0.007182  0.024958
ATT(2004,2003,2005) -0.113422    -0.080509 -0.057046    -0.033583 -0.000051
ATT(2004,2003,2006) -0.140370    -0.108194 -0.080302    -0.052410 -0.020235
ATT(2004,2003,2007) -0.142071    -0.105801 -0.076512    -0.047224 -0.010752
ATT(2005,2002,2003) -0.404228    -0.351068  0.007787     0.366641  0.418972
ATT(2005,2003,2004) -0.406276    -0.349757  0.024788     0.399334  0.452232
ATT(2005,2004,2005) -0.466223    -0.414800 -0.055435     0.303929  0.352599
ATT(2005,2004,2006) -0.615284    -0.545604 -0.114903     0.315799  0.379822
ATT(2005,2004,2007) -0.705760    -0.640085 -0.169502     0.301081  0.364063
ATT(2006,2002,2003)  0.017314     0.033356  0.047702     0.062047  0.078215
ATT(2006,2003,2004) -0.006813     0.007895  0.022110     0.036324  0.051025
```

Conclusion

- `DoubleML` offers a flexible and robust framework for DiD analysis with panel data.
- `DoubleMLDIDMulti` allows estimation of various $ATT(g,t)$'s, accommodating covariates with machine learning.
- Results can be easily aggregated (overall, by group, calendar time, event study) and visualized.
- This enables researchers to conduct nuanced DiD analyses in complex settings with multiple groups and periods.

Further Resources:

- DoubleML Documentation - DiD Models
- DoubleML Example: Panel Data Simple
- DoubleML Example: Panel Data Multiple Periods

Appendix

References

Bach, Philipp, Victor Chernozhukov, Malte S Kurz, and Martin Spindler. 2022. "DoubleML-an Object-Oriented Implementation of Double Machine Learning in Python." *Journal of Machine Learning Research* 23: 53–51.

Bach, Philipp, Malte S. Kurz, Victor Chernozhukov, Martin Spindler, and Sven Klaassen. 2024. "DoubleML: An Object-Oriented Implementation of Double Machine Learning in R." *Journal of Statistical Software*. https://doi.org/10.18637/jss.v108.i03.

Callaway, Brantly, and Pedro HC Sant'Anna. 2021. "Difference-in-Differences with Multiple Time Periods." *Journal of Econometrics* 225 (2): 200–230.

Chang, Neng-Chieh. 2020. "Double/Debiased Machine Learning for Difference-in-Differences Models." *The Econometrics Journal* 23 (2): 177–91.

Chernozhukov, Victor, Christian Hansen, Nathan Kallus, Martin Spindler, and Vasilis Syrgkanis. 2024. *Applied Causal Inference Powered by Machine Learning and AI*. CausalML-book.org. https://causalml-book.org.

Sant'Anna, Pedro HC, and Jun Zhao. 2020. "Doubly Robust Difference-in-Differences Estimators." *Journal of Econometrics* 219 (1): 101–22.

Zimmert, Michael. 2018. "Efficient Difference-in-Differences Estimation with High-Dimensional Common Trend Confounding." *arXiv Preprint arXiv:1809.01643*.